

Отчёт

Проблемы предметной области:

В проекте рассматривается процесс **сборки кроссовок с разными характеристиками**. Каждая модель кроссовок может отличаться:

- Назначением (спорт, город, коллекционные модели)
- Типом подошвы
- Наличием геля для амортизации
- Материалом верха
- Дополнительными коллаборациями с брендами

Проблема заключается в том, что **разные комбинации этих характеристик приводят к множеству вариантов кроссовок**. Если реализовать их создание прямым конструктором (new Sneaker()) с передачей всех параметров), то код становится громоздким, трудным для поддержки и расширения.

В классическом подходе без паттерна:

- Придётся писать отдельные конструкторы для каждого типа кроссовок
- Логика сборки будет дублироваться
- Сложно добавлять новые характеристики (например, гель, премиальный верх или коллаборации)

Решение:

В проекте применён паттерн **Builder**, который разделяет процесс создания объекта и его представления.

Вывод:

Внедрение паттерна облегчает и оптимизирует разработку и дает возможности расширять программу без муторной переписи логики кода

ПЛЮСЫ:

1. **Разделение создания и представления объекта**

- Director отвечает только за последовательность шагов
- Builder отвечает за конкретную реализацию сборки

2. Гибкость и расширяемость

- Можно добавить новые Builder (для других брендов) без изменения Director или GUI
- Легко добавлять новые опции кроссовок

3. Управление сложной логикой

- Каждая комбинация параметров создаётся последовательно
- Легко анимировать каждый шаг сборки

4. Повторное использование кода

- Методы Builder можно переиспользовать для разных типов кроссовок

5. Удобство тестирования и сопровождения

- Можно тестировать отдельные Builder и Director
- GUI остаётся простым и отделённым от логики сборки

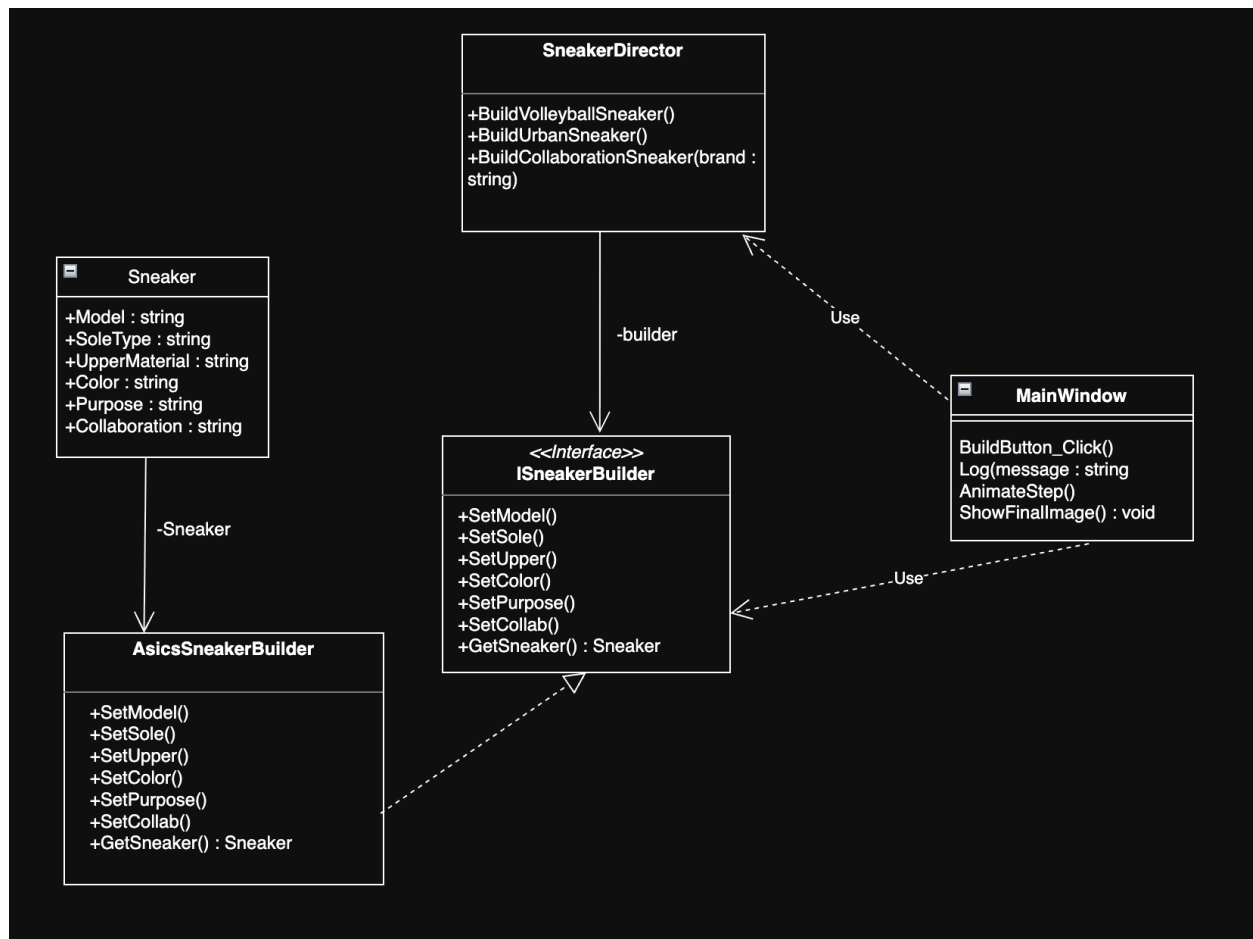


Рис 1: Диаграмма классов конструктора кроссовок

